Guía de estudio del frontend

Esta guía recorre el frontend de Buena Energía desde el arranque de la página hasta las acciones de compra y administración. Está pensada para quien conoce poco Vue o recién empieza a leer este código. Los ejemplos son fragmentos formateados del proyecto: se pueden copiar, buscar y comparar con los archivos originales.

> **Alcance:** la guía explica el frontend. El servidor Laravel y la base de datos son responsables de validar permisos, precios, existencias y pedidos. Las referencias a líneas corresponden a la versión revisada al preparar esta guía y pueden moverse si cambia el código.

1. Qué hace esta aplicación

El frontend es la parte de la tienda que se ejecuta en el navegador. Presenta el catálogo, permite buscar y elegir variantes, guarda una bolsa local y guía al cliente por el checkout. También ofrece una zona de administración para gestionar productos, inventario y pedidos.

La aplicación conversa con el backend por HTTP. Por ejemplo, pide el catálogo, envía un pedido o solicita guardar el stock de una variante. El navegador actualiza la pantalla con las respuestas, pero no es la fuente confiable de los datos comerciales: el backend debe verificar cada operación importante.

2. Mapa del proyecto

```
index.html
  └─ crea el elemento <div id="app">
      └─ src/main.js
          ├─ importa src/style.css
          └─ monta src/App.vue
              └─ src/services/api.js → API del backend Laravel
```

- `index.html` define el documento HTML, el título, la descripción y el punto donde Vue dibuja la aplicación.
- `src/main.js` es la entrada de JavaScript: importa Vue, los estilos y el componente principal, y luego lo monta.
- `src/App.vue` contiene el estado, las acciones y la estructura de las pantallas.
- `src/services/api.js` centraliza las solicitudes HTTP para no mezclar los detalles de Axios con cada botón.
- `src/style.css` define los colores, la distribución, las ventanas, el panel y los ajustes para distintos tamaños de pantalla.
- `vite.config.js` configura Vite y su complemento para Vue.

La aplicación es de página única: `main.js` monta un único `App.vue`. Las distintas secciones y ventanas se muestran u ocultan según el estado; no se usa un sistema de rutas para cambiar entre páginas.

3. Preparar y ejecutar el frontend

Se necesita Node.js 22.18 o posterior y un backend disponible para las funciones que consultan o guardan datos.

Desde PowerShell, en la carpeta `Buena-Energia-Frontend`:

```
npm install
Copy-Item '.env copy.example' .env
npm run dev
```

En `.env`, revisa estas variables:

```
VITE_API_BASE_URL=http://127.0.0.1:8000/api
VITE_WHATSAPP_NUMBER=
```

- `VITE_API_BASE_URL` indica la dirección base de la API. El valor de ejemplo sirve cuando Laravel está disponible en esa dirección.
- `VITE_WHATSAPP_NUMBER` debe contener el número de la tienda en formato internacional, solo dígitos, sin `+`, espacios ni guiones.

Vite lee estas variables al iniciar; reinicia `npm run dev` si cambias `.env`. Las variables con prefijo `VITE_` se incluyen en el código que recibe el navegador: no guardes contraseñas, claves privadas ni la clave de registro de administradores allí.

Para comprobar la compilación de producción:

```
npm run build
npm run preview
```

4. Conceptos de Vue que aparecen en el código

Estado reactivo: `ref`

Una variable normal de JavaScript cambia su valor, pero Vue no necesariamente sabe que debe volver a dibujar la pantalla. `ref` crea un valor reactivo:

```
const searchQuery = ref('')
```

En el código JavaScript se lee o modifica con `.value`:

```
searchQuery.value = 'azul'
```

En el template Vue desenvuelve las referencias automáticamente, por lo que se escribe `{{ searchQuery }}` o `v-model="searchQuery"`, sin `.value`. El estado principal de la tienda está al comienzo de `App.vue`: catálogo, filtros, carrito, checkout, sesión, inventario y pedidos.

Valores derivados: `computed`

Un `computed` representa un resultado calculado a partir de otros datos reactivos. No hace falta mantener el resultado en una segunda variable:

```
const cartCount = computed(() =>
  cartItems.value.reduce((total, item) => total + item.quantity, 0)
)
```

Si una cantidad cambia, Vue vuelve a calcular `cartCount`. El template puede mostrarlo tanto en el botón de bolsa como en el panel. Otros ejemplos son el subtotal (`cartTotal`), los productos filtrados y la cantidad de pedidos pendientes (`App.vue`).

Template: datos, condiciones y listas

El template mezcla HTML con directivas de Vue:

```
<p v-if="isLoading">Cargando...</p>
<p v-else-if="loadError">{{ loadError }}</p>
<article v-for="product in filteredProducts" :key="product.id_producto">
  {{ product.nombre }}
</article>
```

- `{{ valor }}` muestra un valor.
- `v-if` y `v-else-if` deciden qué bloque mostrar.
- `v-for` repite un bloque para cada elemento de una lista.
- `:key` identifica cada elemento de la lista para que Vue actualice el correcto.
- `:atributo="valor"` enlaza un atributo HTML con datos.

El catálogo, por ejemplo, muestra carga, error, productos o un estado vacío, pero no todos a la vez (`App.vue`).

Eventos y formularios

`@click="..."` escucha un clic. `v-model` sincroniza un campo con el estado. `@submit.prevent` escucha el envío de un formulario e impide la recarga normal del navegador:

```
<form @submit.prevent="submitAuth">
  <input v-model.trim="authForm.email" type="email" required>
  <button type="submit">Iniciar sesión</button>
</form>
```

`.trim` elimina espacios al inicio y al final. `required`, `type`, `min`, `maxlength` y otras propiedades HTML también ayudan a validar la entrada. `App.vue` agrega mensajes de validación en español y limpia el mensaje cuando se vuelve a editar el campo (`App.vue`).

Tareas asíncronas: `async` y `await`

Una solicitud al servidor tarda un tiempo. `async` marca una función que puede esperar; `await` pausa esa función hasta recibir el resultado, sin congelar la página:

```
try {
  products.value = await getProducts()
} catch (error) {
  loadError.value = error.message
} finally {
  isLoading.value = false
}
```

`try` contiene el camino normal, `catch` presenta el problema y `finally` siempre apaga el indicador de carga. Es un patrón repetido en catálogo, pedidos y formularios administrativos.

5. Recorrido de inicio: cómo aparece el catálogo

1. `main.js` crea la aplicación Vue y monta `App.vue` en `#app`.
2. `onMounted` se ejecuta cuando el componente aparece por primera vez.
3. La aplicación intenta recuperar la bolsa de `localStorage`.
4. `loadCatalog()` llama a `getProducts()` en `api.js`.
5. La respuesta se guarda en `products`; Vue recalcula filtros y resultados.
6. El template cambia de "cargando" al catálogo, o a un mensaje de error.

La función de carga también reconcilia la bolsa guardada con el catálogo actual: actualiza nombre, precio, variante, stock e imagen cuando puede identificar el producto y la variante. Esto evita que una bolsa persistida use datos viejos. El proceso está en `App.vue` y el inicio del componente en `App.vue`.

6. Ejemplo guiado: buscar y filtrar productos

La búsqueda está construida en dos pasos:

1. `normalizeSearchText()` convierte el texto a minúsculas y quita las marcas de acento. Así, una búsqueda como "basico" puede coincidir con "básico".
2. `filteredProducts` compara las palabras escritas con nombre, descripción, categoría, talle y color; también exige que la categoría seleccionada coincida.

En forma simplificada:

```
const terms = normalizeSearchText(searchQuery.value)
  .trim()
  .split(/\s+/)
  .filter(Boolean)

return products.value.filter((product) =>
  terms.every((term) => searchableText(product).includes(term))
)
```

`every` significa "todas las palabras deben aparecer". Si no hay términos, solo se aplica el filtro de categoría. Como el resultado es un `computed`, se actualiza al escribir sin pedir de nuevo el catálogo al backend (`App.vue`).

7. Ejemplo guiado: agregar a la bolsa

Cuando se pulsa el botón de agregar, `addToCart(product)`:

1. Busca la variante seleccionada para ese producto.
2. Comprueba que la variante exista y tenga stock.
3. Si ya hay una línea para esa variante, comprueba el límite y suma una unidad.
4. Si todavía no hay una línea, crea una con los datos del producto y la variante.
5. Guarda la bolsa en `localStorage` y muestra un aviso visual.

El identificador de variante es importante: dos talles o colores del mismo producto son opciones de inventario distintas. Por eso el carrito evita duplicar líneas de la misma variante y usa su stock para limitar cantidades. El código completo está en `App.vue`.

El subtotal se calcula por línea:

```
subtotal de una línea = precio de la prenda × unidades
subtotal de la bolsa = suma de los subtotales de todas las líneas
```

La bolsa es local hasta que se envía el pedido; editar su contenido no cambia por sí solo el inventario del servidor.

8. Ejemplo guiado: confirmar una compra

El flujo real no consiste solamente en abrir WhatsApp:

```
Bolsa → abrir checkout → pedir CAPTCHA → validar formulario
     → POST /pedidos → recibir pedido confirmado por el backend
     → preparar mensaje → abrir wa.me → vaciar bolsa
```

Al abrir el checkout, `openCheckout()` muestra la ventana y solicita un desafío CAPTCHA. Al enviar, `submitOrder()` comprueba que exista un número de WhatsApp configurado y que el desafío esté listo. Después manda al backend los datos del cliente y entrega, las cantidades e identificadores de variante y los datos del CAPTCHA (`App.vue`).

El cliente no envía el precio como autoridad: el backend debe calcularlo con los datos actuales. Si el servidor rechaza la operación, la ventana temporal de WhatsApp se cierra, se muestra el error, se vuelve a cargar el CAPTCHA y, ante un error de validación, se actualiza el catálogo. La bolsa se conserva.

Solo después de recibir una respuesta exitosa, el frontend arma un resumen con el identificador del pedido, los productos, el total y la entrega. Luego abre el enlace de WhatsApp y limpia la bolsa. La conversación sirve para coordinar la compra; no reemplaza el registro del pedido en el backend.

9. De dónde salen las imágenes

`imageForVariant(product, variantId)` elige la imagen en este orden:

1. Imagen específica de la variante seleccionada.
2. Imagen asociada al color de esa variante.
3. Imagen general del producto.
4. Imagen de respaldo si no hay ninguna de las anteriores.

El orden importa: mostrar la foto de la variante elegida hace que cambiar talle o color no deje una imagen de otra variante. Si un archivo falla al cargarse, `handleImageError` cambia la fuente a la imagen de respaldo (`App.vue`). Las vistas previas de archivos locales se crean con `URL.createObjectURL` y se liberan con `URL.revokeObjectURL` para no dejar recursos temporales ocupados (`App.vue`).

10. Cómo se comunica con la API

`api.js` crea una instancia de Axios con una URL base, un encabezado que espera JSON y un tiempo máximo de espera. Sus funciones representan operaciones con nombres del negocio:

Función del frontend	Operación
`getProducts`	Leer `/productos`.
`getCaptchaChallenge`	Pedir `/captcha/challenge`.
`loginAdministrator` / `registerAdministrator`	Iniciar sesión o registrar una cuenta administrativa.
`createOrder`	Enviar un pedido a `/pedidos`.
`getAdminOrders` / `updateOrderStatus`	Leer pedidos y cambiar su estado.
`createProduct` / `updateProduct`	Crear o editar productos.
`updateVariantStock`	Cambiar el stock de una variante.

Un interceptor adjunta el token guardado en el navegador a las solicitudes cuando hay una sesión. `productFormData()` convierte los datos del producto en `FormData`, necesario para enviar una imagen junto con texto y variantes. La edición incluye los identificadores de las variantes existentes para que el backend pueda distinguirlas de las nuevas.

La capa de API no hace que una operación sea segura por sí sola. El servidor debe autenticar y autorizar cada ruta administrativa y validar los datos aun cuando el frontend ya los haya comprobado.

11. Administración: tres tareas diferentes

Pedidos

`loadAdminOrders()` pide la lista y controla carga y error. `changeOrderStatus()` envía la nueva situación del pedido y actualiza la pantalla con la respuesta. Cuando una transición puede afectar el inventario, se vuelve a solicitar el catálogo (`App.vue`).

Productos

`editProduct(product)` copia los datos existentes al formulario, muestra la pestaña adecuada, espera a que Vue dibuje el formulario y luego desplaza la página hasta él. `submitProduct()` convierte los campos numéricos, prepara las variantes y decide entre crear o actualizar según exista un identificador de edición (`App.vue` y `App.vue`).

Inventario

La tabla de inventario permite escribir un borrador de stock por variante. Al guardar, `saveVariantStock()` comprueba que sea un entero no negativo y pide la actualización al backend. La interfaz solo muestra éxito cuando recibe una respuesta exitosa (`App.vue`).

La sesión habilita o esconde controles en la interfaz, pero eso no constituye un límite de seguridad. Las rutas administrativas deben seguir protegidas en Laravel; nunca se debe confiar en que un botón esté oculto.

12. Estilos, accesibilidad y diseño adaptable

`style.css` organiza el aspecto visual del catálogo, el carrito, los diálogos y el panel. Las reglas `@media` ajustan la distribución para pantallas pequeñas, y la consulta `prefers-reduced-motion` respeta la preferencia del usuario de reducir animaciones.

En el template también aparecen prácticas de accesibilidad:

- Etiquetas asociadas a campos e instrucciones para lectores de pantalla.
- `aria-label`, `aria-expanded` y `aria-pressed` para controles cuyo estado debe comunicarse.
- `role="alert"` para errores y `role="status"` para mensajes de progreso o confirmación.
- Texto alternativo en las imágenes.

Al estudiar un control, sigue su recorrido: busca el elemento en el template, localiza el `@click`, `@submit` o `v-model` y luego sigue el estado que modifica. Por ejemplo, una búsqueda modifica `searchQuery`, que alimenta `filteredProducts`, cuyo resultado se recorre para dibujar las tarjetas.

13. Guía para leer `App.vue` sin perderse

El archivo reúne varias responsabilidades, así que conviene leerlo por partes:

1. **Imports y estado** — qué herramientas y datos usa: `App.vue`.
2. **Validación y valores iniciales** — cómo nacen formularios y mensajes: `App.vue`.
3. **Búsqueda, totales e imágenes** — cálculos derivados: `App.vue`.
4. **Catálogo y bolsa** — carga, agregado, límites y persistencia: `App.vue`.
5. **Checkout** — CAPTCHA, pedido y WhatsApp: `App.vue`.
6. **Administración** — sesión, pedidos, edición e inventario: `App.vue`.
7. **Ciclo de vida** — restauración inicial y limpieza: `App.vue`.
8. **Template** — estructura visual y controles: `App.vue`.

Para encontrar una línea en VS Code, abre el archivo y usa **Ctrl+G**. Si una acción no se entiende, no leas todo el archivo de corrido: busca el nombre del manejador en el template y sigue el estado que cambia.

14. Glosario breve

Término	En palabras simples
API	Puerta por la que frontend y backend intercambian datos.
Backend	Programa del servidor que valida y guarda la información.
Componente	Pieza de interfaz que combina lógica y presentación; aquí, `App.vue`.
Estado	Datos actuales de la aplicación, por ejemplo el texto de búsqueda.
Reactividad	Mecanismo por el cual Vue actualiza la vista cuando cambia el estado.
Variante	Opción concreta de un producto, como un talle y un color, con su stock.
`localStorage`	Espacio del navegador que conserva valores entre recargas.

Término	En palabras simples
`FormData`	Formato de solicitud que permite enviar campos y archivos juntos.
`computed`	Valor calculado que Vue mantiene sincronizado con sus dependencias.
Ciclo de vida	Momentos en que un componente se monta o se desmonta.

15. Errores comunes al probar la aplicación

- **El catálogo no carga:** comprueba que Laravel esté funcionando, que `VITE_API_BASE_URL` apunte al prefijo `/api` correcto y que CORS permita el origen local del frontend.
- **El pedido no continúa a WhatsApp:** configura el número de la tienda y permite ventanas emergentes. El pedido debe registrarse primero en la API.
- **No aparece el panel:** inicia sesión con una cuenta autorizada; ocultar o mostrar el panel en Vue no reemplaza los permisos comprobados por Laravel.
- **Una imagen no se ve:** revisa la URL que devolvió la API y que el backend sirva los archivos cargados.
- **Cambiaste `.env` y no se refleja:** detén y vuelve a iniciar Vite.

16. Comprobación disponible

El proyecto define `npm run build` para validar que el frontend se compile. El `package.json` actual no define un script de pruebas automatizadas.

```
npm run build
```

17. Inventario de archivos: cobertura de esta guía

Este mapa identifica los archivos propios que participan en la aplicación, su configuración, compilación y ejecución. Los ejemplos no reemplazan la lectura del archivo original: abre el enlace para ver la implementación completa.

Inicio, interfaz y comunicación

Archivo	Qué contiene y cómo participa
`index.html`	Documento base que entrega el navegador. Declara metadatos y el elemento `#app` donde Vue dibuja la tienda.
`src/main.js`	Punto de entrada. Importa Vue, los estilos y `App.vue`; crea la aplicación y la monta en `#app`.
`src/App.vue`	Estado reactivo, cálculos, manejadores y template: catálogo, variantes, bolsa, checkout/CAPTCHA, autenticación, pedidos, edición e inventario. La sección 13 divide el archivo por bloques para recorrerlo sin leerlo linealmente.
`src/services/api.js`	Cliente Axios, URL base, timeout, token de administración y funciones para cada endpoint. `productFormData()` serializa campos, variantes e imágenes.

Archivo	Qué contiene y cómo participa
`src/style.css`	Estilos globales de catálogo, panel, formularios, ventanas y mensajes; incluye distribución adaptable y movimiento reducido.

Configuración, comandos y colaboración

Archivo	Qué contiene y cómo participa
`package.json`	Dependencias directas (Vue y Axios), herramientas de desarrollo y comandos `dev`, `build` y `preview`. No define un comando de pruebas.
`vite.config.js`	Activa el complemento oficial de Vue en Vite; es la configuración de compilación del frontend.
`.env_copy.example`	Plantilla local para los nombres de variables que consume Vite. Sus valores no son secretos: no pongas credenciales privadas en variables `VITE_`.
`.gitignore`	Evita agregar a Git archivos locales y salidas generadas, como dependencias, compilación y configuración privada.
`.vscode/extensions.json`	Recomendaciones de extensiones del editor; no cambia el comportamiento de la tienda.
`scripts/clone-buena-energia.ps1`	Script de PowerShell que clona los dos repositorios en un destino. Comprueba Git, valida que una carpeta existente sea el repositorio esperado y admite `-WhatIf`; no sobrescribe un checkout existente.
`README.md`	Requisitos, ejecución local, configuración de integración y resumen de capacidades. Enlaza esta guía y su PDF.

El archivo `.env` local, `package-lock.json`, `node_modules/` y `dist/` no se incluyen en el mapa de lectura: el primero es configuración privada, el lockfile es una resolución de dependencias, las dependencias son código externo y `dist` es salida generada. También se excluye la imagen binaria `public/contacto-buena-energia.jpeg`. Las guías y sus PDF tampoco se cuentan como código fuente.

Un fragmento de inicio con anotaciones

La estructura mínima se puede leer como tres pasos relacionados:

```
import { createApp } from 'vue' // (1) importa la fábrica de aplicaciones de Vue
import './style.css' // (2) carga los estilos globales antes de dibujar la vista
import App from './App.vue' // (3) importa el componente raíz de la tienda

createApp(App).mount('#app') // (4) conecta el componente con index.html
```

Los números señalan el orden de lectura, no una numeración que deba agregarse al archivo real. En `App.vue`, `onMounted()` continúa el flujo: restaura la bolsa guardada y solicita el catálogo. Si la solicitud falla, la vista conserva el mensaje de error y ofrece reintentar; el navegador no trata la bolsa local como inventario confirmado.

Flujo del cliente, visto desde la interfaz

Acción visible	Estado o función de Vue	Solicitud/resultado
Abrir la tienda	`onMounted()` → `loadCatalog()`	`getProducts()` carga productos y variantes; `filteredProducts` recalcula lo que se dibuja.

Acción visible	Estado o función de Vue	Solicitud/resultado
Buscar, filtrar o elegir variante	`searchQuery`, categoría y variante seleccionada	Vue vuelve a calcular tarjetas, imagen y stock disponible sin mutar el servidor.
Agregar o cambiar cantidad	`addToCart()`, `updateQuantity()`, `removeFromCart()`	Se limita la cantidad al stock visto y se persiste la bolsa en `localStorage`; es un borrador local.
Abrir checkout	`openCheckout()` → `loadCaptchaChallenge()`	`getCaptchaChallenge()` obtiene un desafío; el backend conserva la respuesta temporal.
Enviar compra	`submitOrder()`	`createOrder()` envía cliente, entrega, variantes/cantidades y CAPTCHA; el servidor vuelve a validar stock y precio. Solo tras el éxito se genera el resumen de WhatsApp y se vacía la bolsa.
Entrar a administración	`submitAuth()` / `logout()`	`loginAdministrator()` o `registerAdministrator()` guardan la sesión necesaria para el interceptor; salir revoca el token en el backend.
Editar producto	`editProduct()` → `submitProduct()`	Se copian identificadores de variantes existentes; `updateProduct()` usa `FormData` para campos e imagen. Una variante con historial debe conservar su identidad.
Cambiar existencias	`saveVariantStock()`	`updateVariantStock()` solicita el nuevo número; la pantalla muestra el resultado del servidor, no solo el borrador escrito.
Administrar pedido	`loadAdminOrders()` / `changeOrderStatus()`	Se consulta o actualiza el estado. Si la transición cambia existencias, se vuelve a refrescar el catálogo.

La administración de Vue es una capa de interacción, no una barrera de seguridad. El backend debe volver a comprobar token, permisos, datos, stock e historial en cada operación.

18. Alcance y comprobaciones del frontend

El repositorio no tiene archivos de pruebas automatizadas ni un script `test` en `package.json`. Por tanto, `npm run build` demuestra que Vite puede compilar el código, pero no sustituye pruebas de navegador ni verifica reglas del servidor.

Comprobación	Archivo o flujo que conviene recorrer
Compilación y sintaxis de componentes	`npm run build`, `src/App.vue`, `src/main.js` y `vite.config.js`.
Llamadas HTTP y errores de conexión	`src/services/api.js`; catálogo, CAPTCHA, pedido y administración en `src/App.vue`.
Catálogo y filtros	`loadCatalog()`, `normalizeSearchText()`, `filteredProducts` e `imageForVariant()`.
Compra de extremo a extremo	`openCheckout()`, `loadCaptchaChallenge()`, `submitOrder()` y `createOrder()`, además de las pruebas backend de pedido y CAPTCHA.
Edición de producto e imágenes	`editProduct()`, `handleProductImageSelection()`, `submitProduct()` y `productFormData()`; validar persistencia con las pruebas backend de administración.

Comprobación	Archivo o flujo que conviene recorrer
Stock y estados	`saveVariantStock()` y `changeOrderStatus()`; el resultado final depende de `AdminProductoController` y `PedidoController` en el backend.

Los nombres de las funciones permiten saltar desde un evento del template a su manejador. En VS Code, **Ctrl+G** abre una línea y **Ctrl+Shift+F** busca una función por todo el proyecto. Las referencias de línea anteriores son una guía para la versión actual y pueden desplazarse con cambios futuros.

19. Lectura comentada, archivo por archivo

En los capítulos anteriores aparece el panorama funcional. Aquí se recorre el código existente más parecido a leer comentarios al margen: qué recibe cada pieza, qué cambia y a qué archivo pasa el control. Los números de línea son referencias aproximadas a la revisión de esta guía; el comportamiento se confirma en el contenido actual enlazado.

`index.html`: el documento que recibe el navegador

Este HTML es pequeño, pero es el comienzo de toda la aplicación:

1. `<!doctype html>` declara HTML moderno; `lang="es"` anuncia el idioma principal a tecnologías de asistencia y navegadores.
2. `charset` permite interpretar correctamente tildes y otros caracteres. `viewport` hace que el ancho del documento siga el ancho del teléfono.
3. `theme-color` y `description` aportan metadatos de apariencia y resumen; no contienen reglas del negocio.
4. Los dos `preconnect` preparan conexiones a Google Fonts, y el `<link>` descarga DM Sans y DM Serif Display. Sin esa descarga se conservan las familias alternativas del CSS.
5. El `<title>` es el texto de la pestaña del navegador.
6. `<div id="app"></div>` está vacío deliberadamente: Vue lo llena después. `<script type="module" src="/src/main.js">` carga el punto de entrada como módulo ES.

Por eso el identificador `#app` de la última línea de JavaScript debe coincidir con `id="app"` en este archivo. Si se cambia solo uno de los dos, el montaje no encuentra su destino.

`src/main.js`: cuatro líneas que conectan el arranque

`main.js` contiene el mínimo ensamblaje para la SPA:

```
import { createApp } from 'vue' // Función de Vue que crea una instancia de aplicación.
import './style.css' // Los estilos se incluyen en el grafo que Vite compila.
import App from './App.vue' // Componente raíz: su lógica y template forman la tienda.

createApp(App).mount('#app') // Dibuja App dentro del elemento definido en index.html.
```

`createApp(App)` crea la aplicación; `.mount(...)` la hace visible en el documento. El CSS se importa aquí una vez para aplicarse globalmente. No hay router: los enlaces `#catalogo`, `#contacto` y `#historia` son anclas de la misma página, y los paneles se muestran según estado en `App.vue`.

`src/App.vue`: estado, cálculos, acciones y pantallas

El `<script setup>` es la API de composición de Vue dentro de un componente de archivo único. Las variables y funciones declaradas allí quedan disponibles directamente en el `<template>`; no se necesita `return` como en una función `setup()` tradicional. Los imports de `./services/api` son el único puente del componente a la red: los manejadores invocan funciones nombradas y procesan su resultado.

Estado inicial y validación (aprox. líneas 1–141)

- `fallbackImage` proporciona una imagen suplente. `products`, `cartItems`, búsquedas y banderas (`isLoading`, `isCartOpen`) son `ref`: al asignar `.value` en JavaScript, Vue vuelve a representar las partes que dependen de ese dato.
- Los grupos de refs reflejan secciones de interfaz: carrito y mensajes, checkout/CAPTCHA/cliente, autenticación/sesión, formulario e imagen del producto, borradores de stock y pedidos. Esta agrupación mental permite localizar una bandera buscando el nombre de la pantalla.
- `whatsappNumber` lee `VITE_WHATSAPP_NUMBER` al compilar y elimina todo salvo dígitos. Es una configuración pública de contacto, no un lugar para secretos.
- `readStoredUser()` recupera un JSON guardado, y devuelve `null` si no existe o está corrupto. `currentUser` solo decide qué elementos de interfaz mostrar; la autorización verdadera la valida el backend.
- `emptyProductForm()` crea un objeto nuevo y siempre arranca con una variante. Crear un objeto nuevo evita compartir accidentalmente la misma lista entre reinicios del formulario.
- `setSpanishValidationMessage(event)` consulta la API de validación HTML (`validity`) y asigna mensajes locales con `setCustomValidity()`. El manejador complementario borra el mensaje cuando la persona edita. Ambos se conectan una sola vez al contenedor raíz con eventos de captura.

`ref` y `computed` hacen trabajos diferentes. Un `ref` es estado que se cambia; un `computed` es una respuesta derivada. Por ejemplo:

```
const cartCount = computed(() =>
  cartItems.value.reduce((sum, item) => sum + item.quantity, 0)
)
```

Al añadir una línea o variar su cantidad, `cartCount` se recalcula sin que cada botón tenga que actualizar también el contador del encabezado.

Derivaciones y formato (aprox. líneas 142–255)

- `normalizeSearchText()` convierte a texto, descompone caracteres Unicode, quita marcas diacríticas y usa minúsculas en español. Esto permite que `sosten` encuentre `sostén`.
- `categoryOptions` deduce las categorías del catálogo y calcula cuántos productos hay por cada una. No hace una petición; depende de `products`.
- `pendingOrderCount` cuenta pedidos pendientes para la insignia de la pestaña.
- `filteredInventoryProducts` normaliza la búsqueda y filtra producto, talle/color y rango elegido. Crea `inventoryVariants` solo para renderizar la lista, sin reemplazar los datos recibidos.
- `filteredAdminProducts` limita la lista de edición por nombre/categoría.
- `filteredProducts` divide la búsqueda en términos y exige que cada término aparezca en el texto combinado de producto y variantes. Primero aplica categoría; luego busca en nombre, descripción, talle y color.
- `cartCount`, `cartTotal` y `cartLabel` derivan cantidad, importe y etiqueta. El precio aquí es informativo: al crear la orden el backend recalcula el total con su propio catálogo.

- `detailVariant` y `detailImage` siguen el producto y variante elegidos en la ventana de detalles; si no hay producto, la imagen vuelve al respaldo.
- `formatPrice()`, `formatSize()` y `formatCartSize()` solo transforman la presentación; no mutan precio ni talle almacenado.
- `productStockTotal()` suma el stock de las variantes visibles. `imageForVariant()` prefiere la imagen de variante, luego color/producto y finalmente respaldo; `adminProductImage()` da una representación segura para la lista de edición.

Cuando un cálculo aparece en más de un bloque del template, busca si puede ser `computed`: mantener una segunda variable manual sincronizada casi siempre introduce más estados que corregir.

Carga del catálogo y bolsa (aprox. líneas 256–417)

`loadCatalog()` marca carga, limpia el error y llama `getProducts()`. Con la respuesta sustituye `products`, reconcilia cada renglón persistido del carrito con el producto y variante actuales, refresca nombre/precio/stock/imagen y reinicia los borradores de inventario. Si la variante previamente seleccionada desapareció o está agotada, selecciona la primera disponible. Si no hay datos para reconciliar un renglón guardado, lo conserva para no borrarlo en silencio. Un `catch` publica un error entendible; el `finally` apaga el indicador en cualquiera de los dos casos.

`saveCart()` serializa la bolsa a `localStorage`. `showCartFeedback()`, `announceCartNotice()` y `flashAddedVariant()` actualizan avisos y temporizadores visuales. Son feedback, no prueba de que una transacción de servidor se haya completado.

`addToCart(product)` toma el identificador de variante elegido, verifica existencia y stock, y busca esa misma variante en la bolsa. Si existe, aumenta la cantidad solo debajo del stock observado; si no, copia al renglón su identidad, características, precio y URL. Después guarda y anuncia éxito. La clave del carrito es `id_variante`, no `id_producto`: talle/color diferentes pueden tener existencias distintas.

`openProductDetails()` abre el diálogo y elige una variante disponible, respetando primero la selección actual. `addDetailToCart()` sincroniza esa elección con la tarjeta y reutiliza `addToCart()`; así no hay un segundo camino de validación diferente para el botón del diálogo.

`updateQuantity(item, change)` limita incrementos, elimina la línea cuando se llega a cero y persiste el cambio. `removeFromCart(variantId)` elimina directamente esa variante. Ninguna de estas operaciones descuenta inventario remoto.

Checkout, CAPTCHA y pedido (aprox. líneas 418–526)

1. `openCheckout()` abre el modal, limpia errores y lanza `loadCaptchaChallenge()`.
2. `loadCaptchaChallenge()` limpia la respuesta anterior y pide un desafío por `getCaptchaChallenge()`. Si falla, no inventa una respuesta válida.
3. `submitOrder()` comprueba WhatsApp configurado y CAPTCHA listo; abre antes una ventana vacía para no ser bloqueada por el navegador después de esperar la red.
4. `createOrder()` recibe nombre/teléfono, entrega, dirección condicional, notas, identificadores/cantidades y campos CAPTCHA. No envía el precio del carrito como dato de autoridad.
5. Si la API falla, cierra la ventana vacía, traduce los mensajes de validación, pide otro CAPTCHA y vuelve a cargar el catálogo para el caso `422` (por ejemplo, stock/precio desactualizado). Conserva el carrito para corregir y reintentar.
6. Si la API confirma, crea líneas a partir de los detalles devueltos por el servidor y abre el enlace WhatsApp con el ID, total y entrega. Solo entonces la interfaz limpia el carrito.

El punto importante es el orden: el registro en Laravel sucede antes de abrir WhatsApp. Una conversación no equivale a un pedido persistido, y la bolsa del navegador no reserva stock.

Sesión y tareas administrativas (aprox. líneas 527–804)

- `handleImageError()` sustituye una imagen rota por el respaldo y evita seguir usando la URL fallida.
- `openAuth()` prepara el diálogo y modo de autenticación. `submitAuth()` elige login/registro según `authMode`, prepara los campos necesarios, guarda el token y el resumen del usuario en el navegador cuando recibe éxito, y muestra errores del backend. La clave de registro se envía al servidor: no habilita por sí misma la cuenta desde el cliente.
- `logout()` intenta revocar el token y borra sesión local incluso si esa petición falla; cerrar la UI localmente no elimina un token que el servidor no pudo revocar.
- `toggleAdminPanel()` abre/cierra el panel; al abrirlo pide pedidos y catálogo para mostrar información actual. Ocultar un panel no protege los endpoints.
- `toggleInventoryProduct()` cambia solo qué acordeón de producto está abierto.
- `openProductForm()` limpia estado y abre un formulario vacío; si ya está abierto, se interpreta como acción para cerrarlo.
- `editProduct(product)` copia campos e IDs de variante para diferenciar existentes y nuevas, cambia a la pestaña de productos y espera `nextTick()` para que Vue monte el formulario. Luego lo desplaza a la vista y enfoca el nombre; `prefers-reduced-motion` evita animar si se solicitó movimiento reducido.
- `closeProductForm()` descarta el borrador y libera la vista previa.
- `loadAdminOrders()` pide la lista y gestiona carga/error. `changeOrderStatus()` manda la transición, toma el estado devuelto por el servidor, actualiza el catálogo tras confirmar/cancelar (transiciones que tocan existencias) y vuelve a consultar pedidos si hay error.
- `addVariant()` respeta el máximo de 40; `removeVariant()` nunca permite dejar un formulario sin ninguna opción.
- `clearProductImage()` revoca el `blob:` URL temporal y limpia el input. `removeProductImage()` además vacía la imagen existente del formulario. `handleProductImageSelection()` permite solo MIME JPEG/PNG/WebP hasta 5 MB. Estas verificaciones dan feedback rápido, pero el servidor también valida el archivo.
- `submitProduct()` convierte precios y stocks de campos de texto a números, conserva IDs de las opciones editadas y elige `createProduct` o `updateProduct`. Tras éxito cierra el formulario y recarga catálogo; ante `422` presenta los errores de Laravel.
- `saveVariantStock()` solo envía entero no negativo dentro del límite de la columna. Actualiza la variante visible desde la respuesta del servidor y no considera guardado el borrador antes de esa respuesta.

Ciclo de vida y limpieza (aprox. líneas 805–824)

`onMounted()` parsea la bolsa, filtra entradas evidentemente inválidas, la guarda en formato normalizado y llama a `loadCatalog()`. Por eso la vista se monta aunque falle la carga de red: los errores están dentro de la función asíncrona. `onUnmounted()` revoca cualquier URL de imagen y cancela temporizadores activos; evita recursos temporales y callbacks después de desmontar el componente.

Template: cómo se convierte estado en interfaz (aprox. líneas 826–1584)

El árbol visual está organizado por responsabilidad. Los nombres de CSS no son componentes separados: son selectores de `style.css` aplicados al mismo componente.

Bloque del template	Responsabilidad / datos que conecta
Raíz, aviso y encabezado (826–864)	Captura eventos de validez; enlaces de ancla; cambia los botones de cuenta según `isAdministrator`; muestra número de bolsa desde `cartCount`.
Panel de administración (865–1198)	Solo se representa si sesión local y panel están abiertos. Tres pestañas: pedidos (transiciones, datos del cliente y líneas), productos (buscador/editor, archivo/variantes) e inventario (filtros, acordeones y stock). Cada botón llama la acción con nombre correspondiente.
Presentación y catálogo (1200–1330)	Hero, búsqueda/categorías, estados de cargando/error/lista vacía y tarjetas. `v-for` recorre resultados, `:key` identifica cada producto/variante, `v-model` enlaza la selección de talle/color y `@click` delega agregar o ver detalles.
Historia, contacto y pie (1331–1358)	Contenido editorial, imagen estática y navegación al teléfono/contacto; es presentación, no obtiene datos de pedidos.
Diálogo de detalle (1359–1395)	Aparece con `selectedProduct`, actualiza `detailVariantId`, imagen/precio/stock y llama a `addDetailToCart()`.
Acceso de administración (1396–1441)	Login/registro comparten el formulario; `authMode` controla campos condicionales y `submitAuth()` procesa el envío. El input de clave es contraseña para ocultar lo escrito, no porque el navegador la guarde de modo seguro.
Bolsa lateral (1442–1483)	Recorre `cartItems`, actualiza cantidad, elimina líneas, enseña subtotal y abre checkout. `@click.self` cierra solo si el clic fue sobre el fondo, no dentro del panel.
Checkout (1484–1563)	Recoge datos de entrega, CAPTCHA, honeypot y notas; el botón queda ligado a `submitOrder()`. El campo trampa está oculto visualmente pero permanece en el DOM para detectar bots automatizados.
Aviso flotante (1564–1584)	`Transition` muestra mensajes de carrito/confirmación; comunicar `role="status"` permite que tecnologías de asistencia anuncien actualizaciones.

`v-if` crea o retira un bloque; `v-show` lo mantiene y cambia visibilidad. Aquí los modales usan `v-if`, así que su DOM solo existe cuando se abren. `:disabled` evita dobles envíos mientras una tarea está en curso. Los atributos `aria-*`, `role` y `alt` describen controles y estados; no reemplazan etiquetas visibles ni una revisión completa con lector de pantalla.

`src/services/api.js`: contrato HTTP del navegador

El módulo exporta funciones pequeñas que corresponden a operaciones de negocio:

- `axios.create()` fija `baseURL`, `Accept: application/json` y un `timeout` de diez segundos. La variable `VITE_API_BASE_URL` puede sustituir la URL de desarrollo, pero no se deben incluir secretos en variables Vite.
- El interceptor de petición lee el token local y agrega la cabecera de autorización con esquema bearer si existe token. La autorización efectiva la decide el backend, que comprueba el permiso para cada ruta.
- `getProducts()` valida que el cuerpo de respuesta sea una lista y convierte errores de red, timeout o respuesta HTTP en mensajes útiles.
- `getCaptchaChallenge()` y `verifyCaptcha()` llaman a endpoints públicos del CAPTCHA; `loginAdministrator()`, `registerAdministrator()` y `logoutAdministrator()` exponen el flujo de sesión.
- `createOrder()` envía el JSON del pedido. `getAdminOrders()` y `updateOrderStatus()` leen y cambian estados del panel.

- `createProduct()` y `updateProduct()` usan `productFormData()` porque una imagen se manda como archivo junto con texto. El iterador de variantes serializa campos con índices `variantes[n][campo]`; en actualización se incluye `id_variante` para conservar identidad histórica.
- `updateVariantStock()` usa `PATCH` con la cantidad. El backend valida de nuevo todos estos datos: el cliente HTTP es transporte y organización, no una frontera de seguridad.

Ejemplo de uso en `App.vue`: `await getProducts()` devuelve el array de productos. `await createOrder(payload)` devuelve la respuesta de Laravel; si el servidor responde con error Axios lo rechaza y el `catch` de quien llamó puede presentar `error.response.data`.

`src/style.css`: sistema visual y estados de interacción

Este archivo contiene reglas agrupadas por la pantalla a la que visten; los nombres de clase se acuerdan con el template de `App.vue`.

1. `:root` (líneas iniciales) define paleta, familia tipográfica, escalas y sombras reutilizadas. `*` usa `box-sizing` uniforme para que padding y borde se incluyan al calcular tamaños.
2. `.site-shell`, `.announcement-bar`, `.site-header` y navegación estructuran el ancho, franja superior, marca y accesos. `.cart-count-bump` es una animación corta al cambiar el contador.
3. `.hero-*` controla el bloque de presentación, jerarquía tipográfica, recorte de imagen y sello decorativo.
4. `.catalog-*`, `.category-*`, `.search-*`, `.product-*` y `.color-*` disponen filtros y tarjetas; los estados `:hover`, `:focus-visible` y `[aria-pressed="true"]` hacen perceptibles interacción, foco de teclado y categoría seleccionada.
5. `.catalog-message`, `.catalog-error`, `.loading-mark` y `.empty-cart` diferencian carga, error y vacío. No alteran `loadError` ni deciden qué devolverá la API.
6. `.brand-note`, `.contact-*` y `.site-footer` separan contenido editorial, datos de contacto y cierre visual.
7. `.cart-layer` / `.cart-drawer`, `.quantity-control` y `.cart-footer` componen la bolsa lateral. Las clases `drawer-enter-*` sincronizan transiciones de CSS con `<Transition name="drawer">`.
8. `.cart-feedback-*` y `.floating-cart` representan avisos temporales y el acceso persistente a la bolsa; `.checkout-*` y `.delivery-*` dan forma al modal y opciones de entrega.
9. `.product-detail-*` controla el modal de detalle; `.auth-*` el de acceso. `.admin-*`, `.orders-*`, `.inventory-*` y `.product-form-*` se dedican al panel, las tarjetas de pedido, los acordeones de stock y el editor.
10. `@media (min-width: 1500px)`, `(max-width: 900px)` y `(max-width: 620px)` ajustan columnas, márgenes y apilado según el ancho disponible. Las siguientes reglas móviles adaptan cabecera y ventana de acceso. `prefers-reduced-motion: reduce` elimina/reduce animaciones cuando la preferencia del sistema lo solicita.

Al modificar el template, busca el nombre de su clase en este archivo y revisa también sus reglas responsivas. Un elemento puede verse bien a escritorio y romperse dentro de un `@media` si se cambia su estructura sin revisar ambas.

Configuración y archivos auxiliares del frontend

Archivo	Lectura guiada
`package.json`	Declara el proyecto como módulo ES. `dependencies` son Vue y Axios (la aplicación); `devDependencies` son Vite y su complemento Vue (compilación local). `dev` inicia el servidor, `build` genera `dist`, `preview` sirve esa salida. No hay script `test`.

Archivo	Lectura guiada
`vite.config.js`	`defineConfig()` da soporte de editor y declara `vue()` como plugin, necesario para compilar `<script setup>` y los bloques `.vue`. Sin alias o proxy adicional, la URL del backend se configura desde el módulo API.
`.env copy.example`	Nombres de variables de desarrollo: `VITE_API_BASE_URL` y `VITE_WHATSAPP_NUMBER`. Copiarlo a `.env` crea configuración local; Vite incorpora cualquier variable `VITE_` al bundle, por lo que jamás debe guardar una clave privada.
`.gitignore`	Excluye registros, `node_modules`, `dist`, variables `.env` privadas y archivos de editor; permite mantener `.vscode/extensions.json` como recomendación compartida.
`.vscode/extensions.json`	Recomienda Volar para coloreado y soporte de Vue en VS Code; es ayuda del editor, no dependencia de producción.
`scripts/clone-buena-energia.ps1`	Parámetro `Destination` define dónde clonar. `SupportsShouldProcess` habilita `-WhatIf`; luego verifica Git, valida repositorios existentes por `.git` y `origin`, y solo clona destinos ausentes. Si una carpeta existe pero no coincide, lanza error en vez de sobrescribir.
`README.md`	Guía corta de uso para el equipo: requisitos, clonación, variables, API y construcción. Las guías enlazadas aquí explican por qué se siguen esos pasos.
`public/contacto-buena-energia.jpeg`	Imagen estática que Vite copia/expone; archivo binario, no tiene control de flujo ni símbolos a comentar. Se excluye del análisis textual.

No se describe `package-lock.json`: fija el árbol exacto de dependencias para reproducir instalaciones, pero no es código propio; tampoco se repiten `node_modules/`, `dist/`, `.env`, artefactos de PDF ni capturas como si fueran fuente de la aplicación.